WUMIKA Engage

Technical documentation

Customer messaging and retention for the WUMIKA Shopify store: web push, WhatsApp and SMS journeys, campaigns, consent, measurement and privacy.

Version	17 October 2026, from commit 485c5bc 2026-10-15
Scope	Architecture, message flows, every template and journey, configuration, API, data model, security, operations, status
Source	Generated from the repository: templates, journey definitions, settings and endpoints are read from the code

Contents

1. Overview
2. Architecture
3. How a message is sent
4. Policy engine
5. Journeys
6. Campaigns
7. Templates
8. Configuration (settings)
9. Environment and integrations
10. Measurement and analytics
11. Privacy and consent
12. Security
13. Admin console and API
14. Data model
15. Operations
16. Status and pending work

1. Overview

WUMIKA Engage is the store's own messaging system. It listens to what happens on the Shopify store (carts, checkouts, orders, deliveries, payments, browsing) and sends the right customer the right message on the cheapest channel that reaches them: **web push** (free), **WhatsApp** (Meta charges per delivered template message) and **SMS** (DLT-registered, used only as a fallback for order updates and payment links). Email stays with Shopify Email (owner's decision).

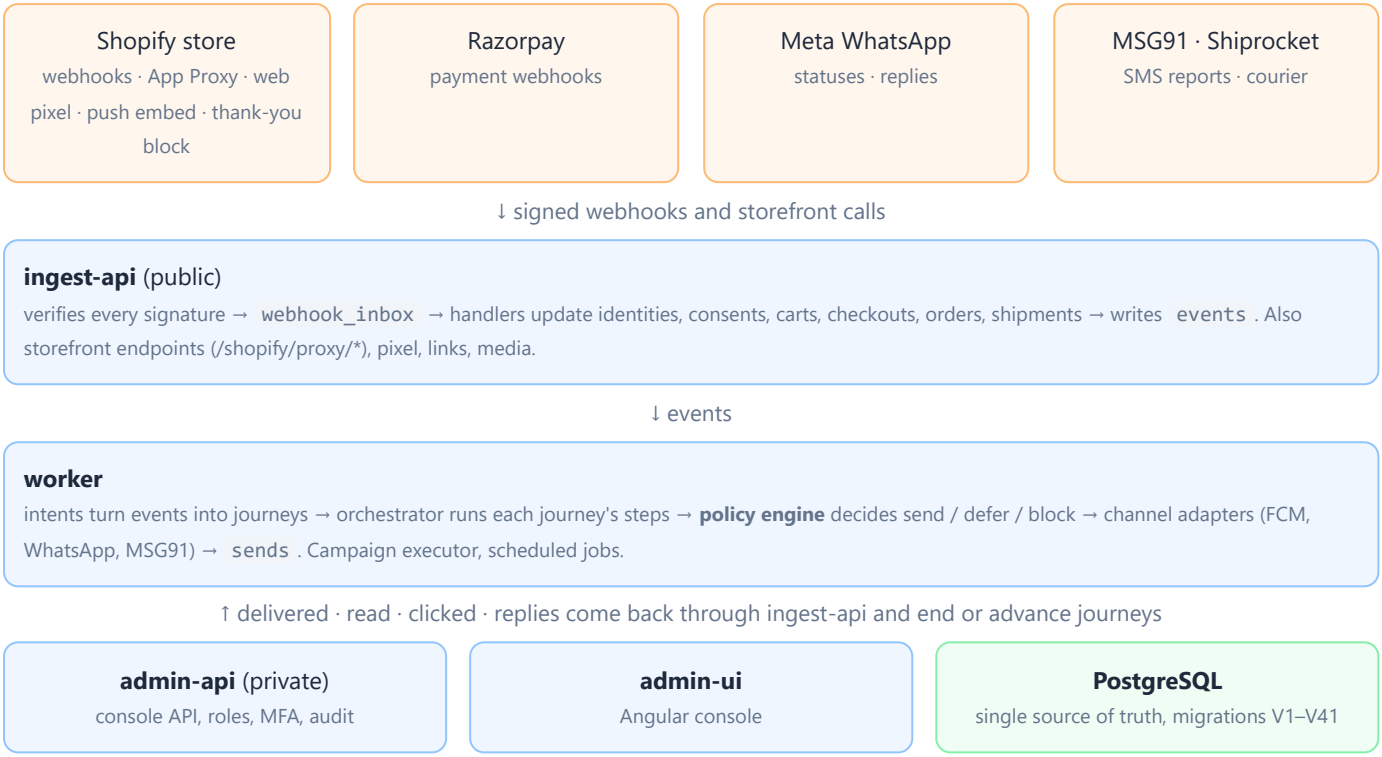
Principles

- **Consent first.** Marketing needs an explicit opt-in per channel; every grant and withdrawal is kept as evidence. STOP on WhatsApp withdraws at once.
- **One conversation per person.** Journeys collide by priority; caps and quiet hours apply across all journeys and campaigns, not per journey.
- **Measured, not assumed.** A share of customers is held out of each journey, so its true lift can be read.
- **Indian D2C practice.** WhatsApp first for high-intent moments, push as the free fallback, offers only at the end and at most once per 60 days per customer, English copy only.
- **Safe by default.** A customer allowlist limits who can be messaged until go-live; providers are never reached from tests.

Channels at a glance

Channel	Used for	Cost	Tracks
Web push (FCM)	Every marketing journey, campaigns, payment retry	Free	Delivered (shown), clicked
WhatsApp (Meta Cloud API)	High-intent reminders, offers, order updates, replies	Marketing ≈ ₹0.86, utility ≈ ₹0.12 per delivered message; replies in the 24 h window free	Delivered, read, replies (button taps are not tracked)
SMS (MSG91, DLT)	Fallback for order updates and the payment link only	Per SMS (DLT)	Delivery reports
Email	Shopify Email (outside Engage)	—	—

2. Architecture



Module	What it does	Technology
ingest-api	Public entry point: Shopify, Razorpay, Meta, MSG91, Shiprocket and SES webhooks; storefront App Proxy endpoints; web pixel intake; signed links (tracking, pay, unsubscribe); poster media and brand icons.	Micronaut 5, Java 25
worker	Event dispatch, journeys (intents), orchestrator ticks, campaign executor, nightly jobs, WhatsApp template sync, spend reconciliation.	Micronaut 5, Java 25
admin-api	Console API: auth (password + TOTP), roles, campaigns, segments, settings, customers, exports, privacy, analytics.	Micronaut 5, Java 25
admin-ui	The console: dashboard, analytics, campaigns, segments, settings, customers, support, privacy, reports.	Angular 22
orchestrator / policy / channels	Libraries: journey runner and templates; the policy engine; FCM, WhatsApp, MSG91 and SES adapters.	Java
shopify-extension	Theme app embed (push opt-in and service worker), web pixel, thank-you page WhatsApp opt-in.	Shopify CLI apps
db/migration	Flyway migrations; ingest-api runs them locally, a migration job in production.	PostgreSQL 16

All money is stored in paise and all times in UTC, shown in IST. Webhooks are stored first and processed after (idempotent by delivery id), so a provider retry or a crash never loses or duplicates an event.

3. How a message is sent

1. **Event.** A webhook or job produces an event (e.g. `checkout_updated`, `order_shipped`, `became_lapsed`).
2. **Intent.** The journey that listens to it decides whether to start, restart or cancel a run for that person and subject (cart, checkout, order, variant).
3. **Cascade run.** The run walks its steps in order. Each step names a channel, a template, conditions (guards) and how long to wait before the next step.
4. **Guards.** A step whose guard fails is skipped and the next one runs at once (e.g. no WhatsApp opt-in → the push goes now).
5. **Policy.** The policy engine decides: send, defer (quiet hours, budget) or block (no consent, cap, holdout, collision). Every decision is stored with its reason.
6. **Send.** The template is rendered (English, with variables) and handed to the channel adapter. Cost is recorded.
7. **Signals.** Delivered, read, clicked, replied and converted come back and either end the run (success) or let it continue.
8. **Credit.** An order is credited to the latest clicked message (7 days) or delivered/read message (24 h), for Analytics.

Step conditions (guards)

Guard	Meaning
`WA_MARKETING`	The person has WhatsApp marketing consent.
`WA_CAPABLE_OR_UNKNOWN`	The number is not known to lack WhatsApp (Meta bills only delivered messages, so trying UNKNOWN costs nothing).
`WA_UNREAD`	No WhatsApp message of this run was read (used before an SMS fallback).
`HAS_LIVE_DEVICE`	At least one active push subscription.
`HAS_EMAIL_MARKETING`	Email marketing consent (email channel is off).
`hasVar(x)` / `lacksVar(x)`	A variable set when the journey started, e.g. `discount`, `wa_ok`, `strong`, `late`.

Priorities

TRANSACTIONAL (payment failed, order updates, replies) > **HIGH_INTENT** (abandoned checkout, back in stock) > **MID_INTENT** (abandoned cart, price drop) > **LOW_INTENT** (browse, win-back, review ask). A lower-priority marketing message is blocked while a higher one is active for the person.

4. Policy engine

Every message, from every journey and campaign, passes these checks in order. The first that fails decides the outcome.

#	Check	Rule
1	**Kill switches**	Marketing halt or a channel halt stops everything at once (read uncached).
2	**Template**	The template exists, is active and matches the channel; the marketing halt applies to marketing templates.
3	**Person**	One read for consent, devices, phone, capability, suppressions.
4	**Consent**	Marketing needs an explicit grant on that channel; WhatsApp needs a WhatsApp opt-in; suppressions block.
4b	**Collisions**	A more important journey active for this person in the last `journey.collision.suppress_hours` (12 h) owns them: HIGHER_PRIORITY_ACTIVE.
5	**WhatsApp window**	A reply inside the 24 h service window is a free service message.
6	**Quiet hours**	Marketing waits until the window opens (a deferral, never a drop). Utility is not held.
7	**Frequency caps**	Across all journeys and campaigns per channel and window (e.g. WhatsApp marketing 1/day, 3/week; push 3/day).
8	**Template cooldown**	Authored in the template YAML (e.g. P1D).
9	**Budget**	WhatsApp spend per IST day; exhaustion defers to the next day.
10	**Holdouts**	Marketing only: the journey holdout (10%) and the global holdout (5%) are measured, never messaged.
—	**Reachability**	Push needs a live device; WhatsApp a phone not INCAPABLE and not in Meta back-off; SMS a phone.

5. Journeys

Defined in code (`worker/…/intents`). Times are from the trigger or from the previous step. Each step's copy is in section 7.

Abandoned checkout `checkout_abandon`

Priority HIGH_INTENT · **Category** Marketing

Starts when: Shopify `checkouts/create|update` with a known buyer (phone or email) and no order. Each update restarts the wait. Starting it ends `cart_recovery` for the same cart.

When	Channel	Template	Condition
t + 30 min stall_minutes	WhatsApp	`checkout_reminder_v1`	Marketing opt-in, number not INCAPABLE, cart ≥ `journey.checkout_abandon.wa_min_cart_paise` . If skipped, the push goes at once.
+ 4 h	Push	`push_checkout_v1`	A live browser subscription. TTL 12 h.
+ 20 h	WhatsApp	`checkout_offer_v1`	As step 1, and a discount applies: the shared code, or (unique mode) a single-use code created in Shopify at send time. At most once per customer per `journey.abandon.discount_cooldown_days` .
+ 12 h	Email	`email_checkout_v1`	Email marketing consent. Off: email stays with Shopify Email (SES built but not configured).

Ends when: Order placed, checkout completed, or a failed payment on the checkout (`payment_failed` takes over). **Success:** Push or WhatsApp click, a WhatsApp reply, or the order (`converted`).

Abandoned cart `cart_recovery`

Priority MID_INTENT · **Category** Marketing

Starts when: Shopify `carts/create|update` for a cart linked to a known person, with no checkout started in the last 24 h. Every change restarts the run; anonymous carts are skipped.

When	Channel	Template	Condition
t + 45 min start_delay_minutes	WhatsApp	`cart_reminder_v1`	Marketing opt-in, not INCAPABLE, cart ≥ `journey.cart_recovery.wa_min_cart_paise` . If skipped, the push goes at once.
+ 4 h	Push	`push_cart_recovery_v1`	TTL 12 h.
+ 20 h	Email	`email_cart_v1`	Email marketing consent (off, as above).

Ends when: Cart emptied, checkout started (checkout journey owns it), order placed. **Success:** Click, reply, or the order.

Browsed, did not add `browse_abandon`

Priority LOW_INTENT · **Category** Marketing

Starts when: Web pixel: 3+ product views in one session, no add-to-cart, 30 minutes idle. Only browsers already linked to a person by the push embed. Each qualifying view restarts the run.

When	Channel	Template	Condition
t + 4 h after last view	Push	`push_browse_abandon_v1`	TTL 24 h. Shows the product photo.
+ 20 h	WhatsApp	`browse_reminder_v1`	Only strong interest (the same piece viewed 2+ times), marketing opt-in, not INCAPABLE, push not clicked.

Ends when: Add to cart, cart change or order (cart recovery takes over); a push click. **Success:** Click.

Price drop `price_drop`

Priority MID_INTENT · **Category** Marketing

Starts when: Shopify `products/update`: a variant's price fell and it is in someone's open cart or on their waitlist. One message per person per drop.

When	Channel	Template	Condition
at once	Push	`push_price_drop_v1`	Real numbers ("Now ₹1,499, was ₹1,899"). TTL 24 h. One-day template cooldown keeps a store-wide markdown to one push a day.
+ 2 h	WhatsApp	`price_drop_v1`	People who did not click the push (or at once if the push cannot reach them). Marketing opt-in, not INCAPABLE.

Ends when: A push click. **Success:** Click.

Back in stock `back_in_stock`

Priority HIGH_INTENT · **Category** Utility (push) / Marketing (WhatsApp)

Starts when: Shopify `inventory_levels/update`: a size variant goes from 0 to in stock. At most quantity × 20 waiting shoppers, oldest first; each is marked notified in the same transaction.

When	Channel	Template	Condition
at once	Push	`push_back_in_stock_v1`	High urgency, TTL 1 h, stale devices allowed (the shopper asked). `push_back_in_stock_nosize_v1` when the variant has no size.
+ 15 min	WhatsApp	`back_in_stock_v1`	Did not click (or at once if no push). Marketing opt-in, not INCAPABLE.

Ends when: A click; the variant sells out again (`variant_sold_out`). **Success:** Click.

Long time no see (win-back) `winback`

Priority LOW_INTENT · **Category** Marketing

Starts when: Nightly 01:30 IST job: the last order is `journey.winback.lapsed_days` old (crossed in the last 7 days) and no storefront activity for `journey.winback.no_visit_days`. Emits `became_lapsed` once per lapse.

When	Channel	Template	Condition
11:00 IST	WhatsApp	`winback_v1`	Marketing opt-in, not INCAPABLE. If skipped, the push goes at once.
+ 7 days	Push	`push_winback_v1`	A live browser subscription.
+ 7 days	WhatsApp	`winback_offer_v1`	Only if a discount is set. Not for customers who returned half of their orders or had 2 RTOs.
+ 3 days	Email	`email_winback_v1`	Off (email stays with Shopify).

Ends when: Any order. **Success:** Click, reply or order.

Payment failed `payment_failed`

Priority TRANSACTIONAL · **Category** Utility

Starts when: Razorpay `payment.failed`. Needs only a WhatsApp opt-in (utility); quiet hours do not apply. Takes the checkout over from `checkout_abandon`.

When	Channel	Template	Condition
t + 5 min first_delay_minutes	WhatsApp	`payment_failed_v1`	Not INCAPABLE; skipped when the webhook arrived 30+ min late.
+ 20 min	Push	`push_payment_retry_v1`	TTL 6 h.
+ 95 min	WhatsApp	`payment_failed_r2_v1`	`payment_failed_r2b_v1` when the shopper cancelled the payment.
+ 4 h	SMS	`sms_payment_link`	Only if no WhatsApp of this run was read. Link /apps/push/pay/<signed ref> redirects to the saved checkout.

Ends when: Captured payment, order, or completed checkout. **Success:** `converted` (the order).

Order updates `order_tracking`

Priority TRANSACTIONAL · **Category** Utility

Starts when: Shopify orders and fulfillments, and Shiprocket courier webhooks. Only for customers with a WhatsApp opt-in; everyone else keeps Shopify's own notifications.

When	Channel	Template	Condition
order placed	WhatsApp	`order_confirmed_v1`	WhatsApp only (Shopify emails the order).
shipped	WhatsApp → SMS	`order_shipped_v1` / `sms_order_shipped`	SMS 2 h later unless read.
out for delivery	WhatsApp → SMS	`out_for_delivery_v1` / `sms_out_for_delivery`	SMS 30 min later unless read.
delivery failed (NDR)	WhatsApp → SMS	`ndr_raised_v1` / `sms_ndr`	SMS 1 h later unless they reply. A reply opens a support request for staff.
refund initiated	WhatsApp	`refund_initiated_v1`	WhatsApp only.
delivered / cancelled	—	—	Nothing sent; ends the parcel's or order's live updates.

Ends when: A later stage supersedes an earlier one; updates over a day old are not sent. INCAPABLE numbers go straight to SMS; errors 131026/131049 escalate to SMS at once. **Success:** Read (or delivered for SMS).

After purchase: "How do you like it?" `post_purchase`

Priority LOW_INTENT · **Category** Marketing

Starts when: `journey.post_purchase.delay_days` (3) after delivery, at 11:00 IST. Once per order; not for cancelled or refunded orders.

When	Channel	Template	Condition
11:00 IST	WhatsApp	`review_ask_v1`	Quick replies "Love it" / "Need help". Marketing opt-in, not INCAPABLE.
on "Love it"	WhatsApp reply	`review_thanks`	Free reply in the window the tap opened; asks for a photo or review.
on "Need help"	WhatsApp reply	`review_help`	Free reply; opens an `after_delivery` support request for staff.

Ends when: Cancellation or refund before it goes. **Success:** A reply.

Delivery reply acknowledgement `delivery_ack`

Priority TRANSACTIONAL · **Category** Service (free)

Starts when: A customer answered a failed-delivery message; ingest-api opened a support request.

When	Channel	Template	Condition
at once	WhatsApp reply	`delivery_ack`	One free reply so they know it reached someone.

Ends when: — **Success:** —

WhatsApp STOP confirmation `wa_stop_confirm`

Priority TRANSACTIONAL · **Category** Service (free)

Starts when: The customer sent STOP (or tapped stop promotions). Their WhatsApp marketing consent is withdrawn first.

When	Channel	Template	Condition
at once	WhatsApp reply	`wa_stop_confirm`	Free-form reply in the window the STOP opened; keeps people from blocking the number.

Ends when: — **Success:** —

Discount codes in journeys

- Abandoned checkout and win-back may offer a code in their last WhatsApp, only if an operator set one.
- **Shared** mode uses the operator's code; **unique** mode creates a single-use code in Shopify at send time (percent, minimum order, valid hours from settings).
- At most once per customer per `journey.abandon.discount_cooldown_days` (60); optionally only for first-time buyers.
- If Shopify refuses, the step is skipped: a customer never receives a code that does not exist.

Support requests

Replies to a failed-delivery message and "Need help" taps open a **support request** that staff handle in the console (update the parcel in Shiprocket, then mark done). Reading the reply text is logged like any look at customer details; it is cleared 30 days after resolution.

6. Campaigns

One-off sends to a segment, written in the console.

1. **Create**: name, channel (push or WhatsApp), template, segment, variables. With `push_campaign_custom_v1` the operator writes the title (40) and text (90) and adds a **poster** (upload JPEG/PNG ≤ 1 MB, or an https link); a phone-style preview shows the expanded Android notification.
2. **Dry run**: real policy decisions for every person in the segment, nothing sent: audience, will receive, excluded and blocked by reason, estimated cost.
3. **Review**: every campaign waits for review. The page shows what goes out (preview, link, variables, audience, cost); the reviewer ticks the checklist before Approve.
4. **Approve**: by an operator with sending rights. One login is enough (the author may approve) unless `approval.four_eyes` is on. Approval re-runs the dry run.
5. **Arm and start**: freezes the audience, snapshots the settings, starts now or at the scheduled time; sends in batches at the set rate.
6. **Running**: pause, resume, cancel; a budget cap stops it. Optional follow-up on the other channel for those who did not click (e.g. WhatsApp 6 h after the push).
7. **Results**: sent, delivered, clicked, orders, revenue and cost; with a holdout, messaged customers against held-back ones.

Halt switches in the console header stop all marketing, or one channel, at once.

7. Templates

Every message is a template in `templates/<channel>/*.yaml`, English only, checked by a linter in the build (length limits, declared variables, banned words, link domains, SMS GSM-7 and DLT rules). Highlighted text is a variable, shown with the sample value the linter uses; real messages carry the actual customer, product and link. **WhatsApp templates are not yet submitted to Meta; SMS DLT IDs are pending.**

Push notifications (11)

Template	Message (with sample values)	Variables
`push_back_in_stock_v1` utility	**Size XXL is back: Floral Anarkali Kurta** Only a few pieces in this restock.	size, product, url, image
`push_back_in_stock_nosize_v1` utility	**Back in stock: Floral Anarkali Kurta** Only a few pieces in this restock.	product, url, image
`push_browse_abandon_v1` marketing · cooldown P1D	**Still thinking about it?** Floral Anarkali Kurta is still in stock. Take another look.	product, url, image
`push_campaign_custom_v1` marketing · cooldown P1D	**Dressed for the Onam festivities ✨** Kasavu sarees, set mundu and festive kurtas: the Onam 2026 edit is here. Shop now.	title, body, url, image
`push_campaign_festive_edit_v1` marketing · cooldown P1D	**The Onam edit is here** Pieces picked for Onam. Explore the edit.	festival, url, image
`push_campaign_new_arrivals_v1` marketing · cooldown P1D	**New in: Handblock Cotton Kurtas** Fresh pieces just landed in Handblock Cotton Kurtas. Take a look.	collection, url, image
`push_cart_recovery_v1` marketing · cooldown PT12H	**Still in your bag: Floral Anarkali Kurta** It's saved for you. Complete your order when you're ready.	product, url, image
`push_checkout_v1` marketing · cooldown PT12H	**Your checkout is saved** Kundan Jhumka Earrings is waiting. Finish your order in a few taps.	product, url, image
`push_payment_retry_v1` utility	**Your payment did not go through** Your WUMIKA order is saved. Tap to try the payment again.	url, image
`push_price_drop_v1` marketing · cooldown P1D	**Price dropped: Floral Anarkali Kurta** Now ₹12,499, was ₹14,999. Same piece, lower price.	product, new_price, old_price, url, image
`push_winback_v1` marketing · cooldown P7D	**New at WUMIKA** Fresh pieces have arrived since your last visit. Take a look.	url, image

WhatsApp (23)

Template	Message (with sample values)	Variables
`back_in_stock_v1` marketing	Good news: Kundan Jhumka Earrings is back in stock at WUMIKA. You asked us to let you know when it returned. View the piece https://wumika.com/products/kundan-jhumka-earrings?variant=44012345678901	product, product_path
`browse_reminder_v1` marketing · cooldown P1D	Still thinking about Kundan Jhumka Earrings? It is still available at WUMIKA, whenever you are ready. View the piece https://wumika.com/products/kundan-jhumka-earrings?variant=44012345678901	product, product_path
`cart_reminder_v1` marketing	Hi Priya, Kundan Jhumka Earrings is still in your WUMIKA bag. Tap below to pick up where you left off. View bag https://wumika.com/cart	name, product
`checkout_offer_v1` marketing	Hi Priya, Kundan Jhumka Earrings is still waiting for you. Use code WUMIKA10 for 10% off on your WUMIKA order. Complete order https://wumika.com/checkouts/cn/hWN4abc123/recover	name, product, code, discount, checkout_path
`checkout_reminder_v1` marketing	Hi Priya, you left Kundan Jhumka Earrings in your WUMIKA checkout. Your details are saved, so finishing your order takes a few seconds. Complete order https://wumika.com/checkouts/cn/hWN4abc123/recover	name, product, checkout_path
`delivered_v1` utility	Your WUMIKA order #1042 has been delivered. Keep your jewellery dry and in its pouch to keep the shine. Reply here if anything needs an exchange.	order_no
`delivery_ack` service	Thanks, Priya. We have passed your message to our delivery team, and they will arrange the next delivery attempt.	name
`exchange_created_v1` utility	Your exchange for WUMIKA order #1042 is booked. Pickup is scheduled for 14 Oct.	order_no, pickup_date
`ndr_raised_v1` utility	Delivery of your WUMIKA order #1042 by Delhivery did not go through: Customer not available at the address. Reply with a convenient time or an updated address and we will arrange another attempt.	order_no, courier, reason
`order_confirmed_v1` utility	Hi Priya, your WUMIKA order #1042 is confirmed. Total ₹12,499. We will message you here when it ships.	name, order_no, amount
`order_shipped_v1` utility	Your WUMIKA order #1042 has shipped with Delhivery (AWB 1234567890123). Track it with the button below. Track order https://wumika.com/apps/push/t/2s-AbCdEfGhIjK	order_no, courier, awb, track_ref
`out_for_delivery_v1` utility	Your WUMIKA order #1042 is out for delivery today with Delhivery. Please keep your phone reachable.	order_no, courier
`payment_failed_v1` utility	Hi Priya, your payment of ₹12,499 for your WUMIKA order did not go through (ref pay_O6gPwJ3yP9rJ3k). Your items are still held for you. You can complete the payment using the button below. Complete payment https://wumika.com/checkouts/cn/abc123/recover	name, amount, payment_id, pay_path

Template	Message (with sample values)	Variables
`payment_failed_r2_v1` utility	Hi Priya, your WUMIKA order is still waiting for payment (ref pay_O6gPwJ3yP9rJ3k). Payments sometimes fail because of the bank or network. You can try again with another UPI app or card. 🔗 Complete payment https://wumika.com/checkouts/cn/hWN4abc123/recover	name, payment_id, pay_path
`payment_failed_r2b_v1` utility	Hi Priya, the payment for your WUMIKA order was cancelled (ref pay_O6gPwJ3yP9rJ3k). Your items are still held if you want to complete it. 🔗 Complete payment https://wumika.com/checkouts/cn/hWN4abc123/recover	name, payment_id, pay_path
`price_drop_v1` marketing · cooldown P1D	Good news: the price of Kundan Jhumka Earrings has dropped to ₹12,499 at WUMIKA (it was ₹14,999). 🔗 View the piece https://wumika.com/products/kundan-jhumka-earrings?variant=44012345678901	product, new_price, old_price, product_path
`refund_initiated_v1` utility	Your refund of ₹12,499 for WUMIKA order #1042 has been started. It will reach your account in 5-7 working days.	amount, order_no, days
`review_ask_v1` marketing	Hi Priya, your Kundan Jhumka Earrings arrived a few days ago. How are you liking it? ↩ Love it (quick reply: review_love) ↩ Need help (quick reply: review_help)	name, product
`review_help` service	Sorry to hear that, Priya. Our team will message you here shortly to help with your order.	name
`review_thanks` service	Thank you, Priya! You can add a photo or a short review at https://wumika.com/products/kundan-jhumka-earrings to help other shoppers choose.	name, review_url
`wa_stop_confirm` service	You will not get more messages from WUMIKA on WhatsApp. Reply START if you change your mind.	—
`winback_v1` marketing	Hi Priya, it has been a while since your last WUMIKA order. We have added new pieces since then, and we think you will like them. 🔗 See what's new https://wumika.com/collections/all	name, link_path
`winback_offer_v1` marketing	Hi Priya, here is something to welcome you back: 10% off on your next WUMIKA order with code WUMK7P2QX9M. 🔗 Shop the collection https://wumika.com/collections/all	name, discount, code, link_path

SMS (DLT) (4)

Template	Message (with sample values)	Variables
`sms_ndr` utility · DLT PENDING	Delivery of your WUMIKA order #1042 by Delhivery did not go through today. Track the next attempt: https://wumika.com/apps/push/t/2s-AbCdEfGhIjK -WUMIKA	order_no, courier, track_ref
`sms_order_shipped` utility · DLT PENDING	Your WUMIKA order #1042 has shipped with Delhivery. Track it here: https://wumika.com/apps/push/t/2s-AbCdEfGhIjK -WUMIKA	order_no, courier, track_ref
`sms_out_for_delivery` utility · DLT PENDING	Your WUMIKA order #1042 is out for delivery today. Track it here: https://wumika.com/apps/push/t/2s-AbCdEfGhIjK -WUMIKA	order_no, track_ref

Template	Message (with sample values)	Variables
`sms_payment_link` utility · DLT PENDING	Your payment of Rs 12,499 to WUMIKA did not go through (ref pay_O6gPwJ3yP9rJ3k). Complete it here: https://wumika.com/apps/push/pay/2s-AbCdEfGhIjK -WUMIKA	amount_plain, payment_id, pay_ref

Email (built, off) (3)

Template	Message (with sample values)	Variables
`email_cart_v1` marketing	**Still thinking about Kundan Jhumka Earrings?** Hi Priya, Kundan Jhumka Earrings is still in your WUMIKA bag. We have kept it there for you. Take another look whenever you are ready. View your bag https://wumika.com/cart	name, product, url
`email_checkout_v1` marketing	**Your WUMIKA checkout is saved** Hi Priya, You were a few steps away from ordering Kundan Jhumka Earrings. Your details are saved, so it only takes a moment to finish. Complete your order https://wumika.com/checkouts/cn/hWN4abc123/recover	name, product, url
`email_winback_v1` marketing	**It has been a while, Priya** Hi Priya, We have added new pieces since your last WUMIKA order, and we would love to show you what is new. See what's new https://wumika.com/collections/all	name, url

Push extras: every push carries the brand W icon and status-bar badge (or `PUSH_ICON_URL` / `PUSH_BADGE_URL`), journey pushes carry the product photo (Shopify CDN, 1024 px), campaigns may carry a poster. Images show on Chrome and Edge (Android, Windows); iPhone and Safari show text only. No live countdown exists in web push: put the end time in the text.

8. Configuration (settings)

Business settings live in the database and are changed in the console (Settings), never by deploying. Every change is versioned, effective-dated, audited, and may carry an end date (e.g. a festive cap that reverts on its own). **CRITICAL** settings go through a proposal that is approved (one login may approve its own while `approval.four_eyes` is off). Values below are the defaults.

Kill switches

Key	Meaning	Default	Scope	Risk
`halt.channel`	Halt channel Stops every send on this channel, utility included. Use during a provider incident.	`false`	CHANNEL	GUARDED
`halt.marketing`	Halt all marketing Stops marketing on every channel. Utility continues.	`false`	GLOBAL	GUARDED
`halt.journey`	Halt journey	`false`	JOURNEY	GUARDED

Quiet_Hours

Key	Meaning	Default	Scope	Risk
`quiet_hours`	Quiet hours (IST) Marketing is deferred to the next open window. Order-critical utility templates are exempt.	`{"to": "09:00", "from": "21:00"}`	GLOBAL	GUARDED

Frequency caps

Key	Meaning	Default	Scope	Risk
`cap.whatsapp.marketing.1d`	WhatsApp marketing / day Keep below Meta's own per-user limit so you never hit it.	`1`	CHANNEL	CRITICAL
`cap.whatsapp.marketing.7d`	WhatsApp marketing / week	`3`	CHANNEL	CRITICAL
`cap.push.marketing.1d`	Push / day	`3`	CHANNEL	GUARDED
`cap.email.marketing.7d`	Email / week	`3`	CHANNEL	GUARDED

Budgets

Key	Meaning	Default	Scope	Risk
`budget.whatsapp.marketing.daily_paise`	Daily WhatsApp marketing budget (paise) Hard stop. A runaway journey costs you a day, not a quarter.	`200000`	CHANNEL	CRITICAL

Provider rates

Key	Meaning	Default	Scope	Risk
`rate.whatsapp.marketing_paise`	Meta marketing rate (paise) Used for pre-send estimates. Reconcile against Meta billing monthly.	`86`	CHANNEL	GUARDED
`rate.whatsapp.utility_paise`	Meta utility rate (paise)	`12`	CHANNEL	GUARDED

Key	Meaning	Default	Scope	Risk
`rate.sms.utility_paise`	SMS transactional rate (paise) MSG91 price per 160-character part, DLT scrubbing included. Verify against your MSG91 rate card.	`20`	CHANNEL	GUARDED

Holdouts

Key	Meaning	Default	Scope	Risk
`holdout.global_pct`	Global holdout % Never measured, never messaged. Keep this on permanently.	`5.0`	GLOBAL	CRITICAL
`holdout.journey_pct`	Journey holdout % Share of identities held out of this journey to measure its lift.	`10`	JOURNEY	CRITICAL

Journeys

Key	Meaning	Default	Scope	Risk
`journey.enabled`	Journey enabled	`true`	JOURNEY	GUARDED
`journey.payment_failed.first_delay_minutes`	Payment failed: first message after (min) Most UPI failures are retried at once. Too early reads as the brand not knowing.	`5`	JOURNEY	GUARDED
`journey.checkout_abandon.stall_minutes`	Checkout reminder after (min) Minutes after the last checkout activity before the first reminder. Any later activity restarts the wait.	`30`	JOURNEY	GUARDED
`journey.checkout_abandon.wa_min_cart_paise`	Checkout reminder: WhatsApp min cart (paise) Below this, the checkout reminders go by push only. 0 = every cart. A marketing WhatsApp costs about ₹0.86 when delivered.	`0`	JOURNEY	CRITICAL
`journey.cart_recovery.start_delay_minutes`	Cart reminder after (min) Minutes after the last cart change. Every change restarts the wait.	`45`	JOURNEY	GUARDED
`journey.cart_recovery.wa_min_cart_paise`	Cart reminder: WhatsApp min cart (paise) Below this, the cart reminder goes by push only. 0 = every cart.	`0`	JOURNEY	CRITICAL
`journey.winback.lapsed_days`	Win-back: days since last order Jewellery is bought every few months; 90 days suits most customers. The message goes once per lapse.	`90`	JOURNEY	GUARDED
`journey.abandon.discount_mode`	Abandoned checkout: discount codes shared: the one code set below. unique: a single-use code per customer, made by the platform in Shopify (needs write_discounts on the app).	`shared`	JOURNEY	CRITICAL
`journey.collision.suppress_hours`	Quiet time after a more important journey (h) After a checkout or back-in-stock journey ends, cart, browse and win-back messages wait this long. Order updates are never held.	`12`	GLOBAL	GUARDED
`journey.abandon.discount_code`	Abandoned checkout: discount code A code created in Shopify admin (Discounts). Sent in the last WhatsApp reminder, about 24 h after the checkout. Empty = no discount message.		JOURNEY	CRITICAL

Key	Meaning	Default	Scope	Risk
`journey.abandon.discount_label`	Abandoned checkout: what the code gives Shown with the code, e.g. "10% off". Must match the code in Shopify.		JOURNEY	CRITICAL
`journey.abandon.discount_cooldown_days`	Abandoned checkout: discount at most once per (days) A customer who got the discount message gets no other within this many days. The reminders still go.	`60`	JOURNEY	GUARDED
`journey.abandon.discount_new_customers_only`	Abandoned checkout: discount for first-time buyers only On: customers with a past order get the reminders but never the discount.	`false`	JOURNEY	GUARDED
`journey.abandon.discount_percent`	Unique codes: percent off Percentage off the whole order for each single-use code.	`10`	JOURNEY	CRITICAL
`journey.abandon.discount_min_order_paise`	Unique codes: minimum order (paise) The code only works on an order of at least this subtotal. 0 = no minimum.	`0`	JOURNEY	CRITICAL
`journey.abandon.discount_valid_hours`	Unique codes: valid for (hours) How long each code works after the message is sent.	`48`	JOURNEY	GUARDED
`journey.winback.no_visit_days`	Win-back: only if no visit for (days) Someone browsing recently gets the browse and cart messages instead.	`30`	JOURNEY	GUARDED
`journey.winback.link_path`	Win-back: page the message opens A path on the store, e.g. collections/new-arrivals. collections/all always exists.	`collections/all`	JOURNEY	SAFE
`journey.winback.discount_mode`	Win-back: discount codes shared: the one code set below. unique: a single-use code per customer, made in Shopify (needs write_discounts).	`shared`	JOURNEY	CRITICAL
`journey.winback.discount_code`	Win-back: shared discount code Created in Shopify admin. Empty (and shared mode) = no offer message.		JOURNEY	CRITICAL
`journey.winback.discount_label`	Win-back: what the shared code gives e.g. "10% off". Must match the code in Shopify.		JOURNEY	CRITICAL
`journey.winback.discount_percent`	Win-back unique codes: percent off	`10`	JOURNEY	CRITICAL
`journey.winback.discount_min_order_paise`	Win-back unique codes: minimum order (paise) 0 = no minimum.	`0`	JOURNEY	CRITICAL
`journey.winback.discount_valid_hours`	Win-back unique codes: valid for (hours)	`48`	JOURNEY	GUARDED
`journey.post_purchase.delay_days`	After delivery: "how do you like it?" after (days) Long enough to have worn it, soon enough to remember. Once per order.	`3`	JOURNEY	GUARDED

Approvals

Key	Meaning	Default	Scope	Risk
`approval.required_above_paise`	Second approver required above (paise) (deprecated) No longer used (2026-10-15): every campaign is reviewed; approval.four_eyes decides who may approve.	`1000000`	GLOBAL	CRITICAL
`approval.four_eyes`	Second approver for campaigns and critical settings Off: one login is enough; you review and approve your own campaigns and critical setting changes. On: someone else must approve them.	`false`	GLOBAL	GUARDED

Revenue credit

Key	Meaning	Default	Scope	Risk
`attribution.click_days`	Revenue credit: after a click (days) An order is credited to the latest message clicked within this many days before it.	`7`	GLOBAL	GUARDED
`attribution.view_hours`	Revenue credit: after a delivered or read message (hours) Without a click, an order is credited to the latest message delivered or read within this many hours. 0 = clicks only.	`24`	GLOBAL	GUARDED

Push

Key	Meaning	Default	Scope	Risk
`push.ttl_seconds.default`	Push TTL default (s)	`43200`	CHANNEL	SAFE
`push.campaign_stale_days`	Push: token stale after (days) Stale tokens are excluded from campaigns, kept for back-in-stock.	`30`	CHANNEL	GUARDED

WhatsApp

Key	Meaning	Default	Scope	Risk
`wa.service_window_hours`	WhatsApp service window (h) Meta: 24.	`24`	CHANNEL	SAFE
`wa.backoff_131049_hours`	Back off after 131049 (h) Meta chose not to deliver a marketing message: skip WhatsApp for this person this long.	`24`	CHANNEL	GUARDED
`wa.incapable_recheck_days`	Re-check INCAPABLE after (days) People install WhatsApp; a number written off is tried again after this.	`120`	CHANNEL	GUARDED

9. Environment and integrations

Secrets and endpoints come from environment variables (the platform secret manager in production; `/opt/wumika/.env` on the dev server; git-ignored `config/local.env` locally). They are never in the database, the console or the repository.

Variable	Example	Notes
`DB_NAME`	`wumika_dev`	
`DB_USER`	`engage`	
`DB_PASSWORD`	secret	openssl rand -hex 24
`CUSTOMER_ALLOWLIST_EMAILS`	`marylindajohnson22@gmail.com`	
`SHOPIFY_SHOP_DOMAIN`	`CHANGE_ME.myshopify.com`	
`STOREFRONT_BASE_URL`	secret	Where push links point (cart_recovery -> <this>/cart). The domain the shopper subscribed on; blank falls back to https://SHOPIFY_SHOP_DOMAIN.
`SHOPIFY_API_SECRET`	secret	
`SHOPIFY_CLIENT_ID`	`CHANGE_ME`	
`SHOPIFY_ADMIN_TOKEN`	secret	Optional: Admin API tokens are fetched automatically with the client ID and secret above (client credentials grant). Leave blank unless the store is outside the app's Dev Dashboard organization.
`SHOPIFY_ADMIN_API_VERSION`	`2026-07`	
`PIXEL_WRITE_KEY`	secret	Web pixel intake (P2-T06); same value as the pixel's webPixelCreate settings. Blank = /pixel/events answers 503.
`METRICS_TOKEN`	secret	Bearer token for ingest-api /prometheus (P1-T05). Blank = /prometheus is 404.
`CHECKOUT_NOTICE_SINCE`	secret	Date the checkout phone label started showing checkout_notice_v1's text (checkout_notice_v2: "Phone (for order updates and offers on WhatsApp/SMS)"), e.g. 2026-10-01. Orders from then get WhatsApp order updates and offers on that notice (opt-out; STOP always wins). Blank = off.
`CHECKOUT_NOTICE_VERSION`	`checkout_notice_v2`	The live label text: checkout_notice_v2 (default) or checkout_notice_v1 (order updates only).
`COURIER_WEBHOOK_TOKEN`	secret	Shiprocket tracking webhook (P1-T02, ADR-005): invent a long random value and paste it as the security token in Shiprocket -> Settings -> API -> Webhooks (URL: https://<ingest host>/webhooks/courier). Blank = refused.
`RAZORPAY_WEBHOOK_SECRET`	secret	
`RAZORPAY_MODE`	`test`	
`RAZORPAY_MATCH_WINDOW_MINUTES`	`30`	
`PUBLIC_BASE_URL`	`https://engage-dev.wumika.com`	

Variable	Example	Notes
`PUSH_ICON_URL`	secret	Push notifications: the brand logo beside every notification (https, square, 192 px) and Android's status-bar icon (https, white on transparent, 96 px). Not secret. Blank = the W icons shipped at PUBLIC_BASE_URL/brand/. A Shopify Files URL works.
`PUSH_BADGE_URL`	secret	
`BUSINESS_TZ`	`Asia/Kolkata`	
`ADMIN_MFA_KEY`	secret	openssl rand -hex 32
`ADMIN_BOOTSTRAP_EMAIL`	`CHANGE_ME`	

Other variables

Variable	Purpose
`WA_ACCESS_TOKEN, WA_APP_SECRET, WA_PHONE_NUMBER_ID, WA_BUSINESS_ACCOUNT_ID, WA_WEBHOOK_VERIFY_TOKEN, WA_GRAPH_VERSION`	WhatsApp Cloud API: permanent system-user token, app secret (webhook signatures), number, account, webhook verification.
`MSG91_AUTH_KEY, MSG91_DLR_TOKEN`	SMS sending and the secret in the delivery-report URL.
`COURIER_WEBHOOK_TOKEN`	Shiprocket webhook secret (in its URL).
`RAZORPAY_WEBHOOK_SECRET, RAZORPAY_MODE, RAZORPAY_MATCH_WINDOW_MINUTES`	Razorpay webhook signatures; test or live; phone-and-amount matching window.
`SHOPIFY_SHOP_DOMAIN, SHOPIFY_CLIENT_ID, SHOPIFY_API_SECRET`	The store, and the app's credentials (webhook and App Proxy signatures, Admin API, signed links).
`CUSTOMER_ALLOWLIST_EMAILS`	Who may be messaged: a list of test emails until go-live, `*` after.
`CHECKOUT_NOTICE_SINCE, CHECKOUT_NOTICE_VERSION`	When the checkout phone notice went live (basis for order updates).
`PUBLIC_BASE_URL, PUSH_ICON_URL, PUSH_BADGE_URL`	Public address (posters, icons, links); optional brand icon overrides.
`ADMIN_JWT_KEY_FILE, ADMIN_MFA_KEY, ADMIN_BOOTSTRAP_EMAIL`	Console token keys, MFA secret encryption, first Owner.
`METRICS_TOKEN, PIXEL_WRITE_KEY`	Prometheus endpoint; web pixel write key.
`FIREBASE_SERVICE_ACCOUNT_FILE`	FCM service account for push.
`EMAIL_FROM, SES_*, AWS_*`	Email through SES (built, off).
`FLYWAY_ON_STARTUP, WORKER_JOBS_ENABLED`	Run migrations at start (local only); turn scheduled jobs off.

Integration endpoints (to configure at each provider)

Provider	Endpoint on ingest-api	Signature
Shopify webhooks	`/webhooks/shopify`	X-Shopify-Hmac-Sha256 (HMAC-SHA256, app secret)
Shopify App Proxy	`/apps/push/*` → `/shopify/proxy/*`	Query signature (app secret)

Provider	Endpoint on ingest-api	Signature
Shopify compliance	`/webhooks/shopify` (customers/redact, customers/data_request, shop/redact)	Same HMAC
Razorpay	`/webhooks/razorpay`	X-Razorpay-Signature
Meta WhatsApp	`/meta/webhook` (GET verify, POST events)	X-Hub-Signature-256 (app secret)
MSG91 delivery reports	`/webhooks/sms?token=…`	URL token
Shiprocket	`/webhooks/courier?token=…`	URL token
Web pixel	`/pixel/events`	X-Engage-Key
Thank-you page opt-in	`/shopify/thankyou/whatsapp-optin`	Shopify session token (JWT)

10. Measurement and analytics

Revenue credit (attribution)

When an order arrives it is credited, once, to the latest message the customer **clicked** within `attribution.click_days` (7), otherwise to the latest message **delivered or read** within `attribution.view_hours` (24) — the way WhatsApp, whose button taps are not tracked, gets credit. Order updates, delivery acknowledgements and the STOP confirmation are never credited. Revenue is the order total less refunds; cancelled orders are left out.

Holdouts and lift

Each marketing journey holds out `holdout.journey_pct` (10%) of customers and all marketing holds out `holdout.global_pct` (5%). Reports compares revenue per person between messaged and held-out groups with Welch's t-test and a 95% range, needs 4 weeks and 30 people per group, and only then calls a difference real. This is the honest measure of what a journey adds.

Console reports

- **Dashboard**: today's spend against budget, last 24 h by status and block reason, journey health, opt-ins and opt-outs, push subscribers, WhatsApp templates.
- **Analytics**: for 7/30/90 days or custom dates — totals, day-by-day chart, funnel by channel and by journey (sent, delivered, read, clicked, orders, revenue, cost, return per ₹1), campaigns, spend and block reasons, WhatsApp quality.
- **Campaign results**: per campaign funnel, revenue, cost, held back vs messaged.
- **Reports**: journey lift; WhatsApp spend against Meta's monthly bill (CSV upload), with a daily check that alerts over 1%.

11. Privacy and consent

- **Consent ledger**: append-only; every grant and withdrawal keeps the exact copy shown, page, time and source. Sources: checkout and cart opt-ins, thank-you page, push soft-ask, Shopify email consent, WhatsApp STOP/START.
- **Allowlist** until go-live; synthetic data in tests; providers never reached from tests.
- **PII access**: contact details are masked in the console; revealing one, reading a support reply or building a data report is logged and limited to 50 a day per operator.
- **Erasure (DPDP)**: `erase_identity()` deletes the person and everything merged into them, clears phone and email on orders, checkouts and payments, scrubs raw webhook payloads, courier events, anonymous pixel events and export files, keeps an anonymous proof of each withdrawal, and moves spend to a tombstone identity so totals still add up. From the console (Owner, with a reason) or Shopify's `customers/redact`.
- **Data requests**: Shopify's `customers/data_request` opens a privacy request; the Owner downloads a JSON report built on the spot and marks it answered.
- **Shop redact**: alert and a manual runbook (never automatic).

12. Security

- **Console sign-in**: password (Argon2id) plus TOTP for roles above Analyst; recovery codes; lockout after 5 failures; per-IP limit; short-lived JWTs with refresh rotation.
- **Roles**: VIEWER, ANALYST, CAMPAIGN_EDIT, CAMPAIGN_SEND, CONFIG_ADMIN, OWNER. Every endpoint declares its role (a build test enforces it).
- **Webhooks**: every provider's signature verified before storage; inbox deduplicated by delivery id.
- **Storefront**: App Proxy signature binds the logged-in customer; signed links for tracking, pay and unsubscribe; rate limits shared across servers in Postgres; at most 50 waiting back-in-stock alerts per shopper.
- **Errors**: unexpected failures answer a plain 500 with a reference; details stay in the log (no customer data in responses). A seeded fuzz test covers every public endpoint.
- **Audit log**: append-only record of every operator action, approval, setting change and erasure.
- Pen-test scope, credential rotation list and backup rehearsal: `docs/qa/p7-security.md`.

13. Admin console and API

The console (admin-ui) calls admin-api. Main screens: Dashboard, Analytics, Reports, Campaigns, Segments, Journey inspector, Customers, Support requests, Settings, Templates, Consent copy, Payment failures, Ingest health, Exports, Privacy requests, Operators, Account.

Service	Method	Path	Role
admin-api	GET	`/.well-known/jwks.json`	public / signed
admin-api	GET	`/api/analytics/campaigns`	VIEWER
admin-api	GET	`/api/analytics/overview`	VIEWER
admin-api	POST	`/api/auth/login`	public / signed
admin-api	POST	`/api/auth/logout`	public / signed
admin-api	GET	`/api/auth/me`	VIEWER
admin-api	POST	`/api/auth/mfa`	public / signed
admin-api	POST	`/api/auth/mfa/confirm`	public / signed
admin-api	POST	`/api/auth/mfa/enrol`	public / signed
admin-api	POST	`/api/auth/mfa/recovery-codes`	VIEWER
admin-api	POST	`/api/auth/refresh`	public / signed
admin-api	POST	`/api/auth/set-password`	public / signed
admin-api	GET	`/api/campaigns`	VIEWER
admin-api	POST	`/api/campaigns`	CAMPAIGN_EDIT
admin-api	GET	`/api/campaigns/templates`	VIEWER
admin-api	GET	`/api/campaigns/{id}`	VIEWER
admin-api	PATCH	`/api/campaigns/{id}`	CAMPAIGN_EDIT
admin-api	POST	`/api/campaigns/{id}/approve`	CAMPAIGN_SEND
admin-api	POST	`/api/campaigns/{id}/cancel`	CAMPAIGN_SEND
admin-api	POST	`/api/campaigns/{id}/estimate`	CAMPAIGN_EDIT
admin-api	POST	`/api/campaigns/{id}/pause`	CAMPAIGN_SEND
admin-api	GET	`/api/campaigns/{id}/report`	VIEWER
admin-api	POST	`/api/campaigns/{id}/resume`	CAMPAIGN_SEND
admin-api	POST	`/api/campaigns/{id}/start`	CAMPAIGN_SEND
admin-api	GET	`/api/config`	VIEWER
admin-api	DELETE	`/api/config/proposals/{id}`	CONFIG_ADMIN
admin-api	POST	`/api/config/proposals/{id}/approve`	CONFIG_ADMIN
admin-api	POST	`/api/config/proposals/{id}/reject`	CONFIG_ADMIN

Service	Method	Path	Role
admin-api	GET	`/api/config/snapshots/{id}`	VIEWER
admin-api	POST	`/api/config/{key}`	CONFIG_ADMIN
admin-api	GET	`/api/config/{key}/history`	ANALYST
admin-api	GET	`/api/consent-copy`	VIEWER
admin-api	POST	`/api/consent-copy`	CONFIG_ADMIN
admin-api	GET	`/api/customers`	VIEWER
admin-api	GET	`/api/customers/{id}`	VIEWER
admin-api	POST	`/api/customers/{id}/erase`	OWNER
admin-api	POST	`/api/customers/{id}/reveal`	ANALYST
admin-api	GET	`/api/dashboard`	VIEWER
admin-api	GET	`/api/exports`	ANALYST
admin-api	POST	`/api/exports`	ANALYST
admin-api	GET	`/api/exports/download`	public / signed
admin-api	POST	`/api/exports/{id}/link`	ANALYST
admin-api	GET	`/api/halt`	VIEWER
admin-api	POST	`/api/halt`	public / signed
admin-api	DELETE	`/api/halt/{scope}/{selector}`	CONFIG_ADMIN
admin-api	GET	`/api/ingest/health`	VIEWER
admin-api	POST	`/api/inspector`	ANALYST
admin-api	GET	`/api/journeys/{key}/incrementality`	ANALYST
admin-api	GET	`/api/media`	CAMPAIGN_EDIT
admin-api	POST	`/api/media`	CAMPAIGN_EDIT
admin-api	GET	`/api/operators`	OWNER
admin-api	POST	`/api/operators`	OWNER
admin-api	POST	`/api/operators/{id}/disable`	OWNER
admin-api	POST	`/api/operators/{id}/reset`	OWNER
admin-api	PUT	`/api/operators/{id}/roles`	OWNER
admin-api	GET	`/api/payments/failures`	VIEWER
admin-api	GET	`/api/privacy-requests`	OWNER
admin-api	POST	`/api/privacy-requests/{id}/report`	OWNER
admin-api	POST	`/api/privacy-requests/{id}/resolve`	OWNER
admin-api	GET	`/api/reports/blocks`	VIEWER

Service	Method	Path	Role
admin-api	GET	`/api/reports/lift`	ANALYST
admin-api	POST	`/api/reports/meta-billing`	CONFIG_ADMIN
admin-api	GET	`/api/reports/quality`	VIEWER
admin-api	GET	`/api/reports/spend`	VIEWER
admin-api	GET	`/api/reports/spend-reconciliation`	VIEWER
admin-api	GET	`/api/segments`	VIEWER
admin-api	POST	`/api/segments`	CAMPAIGN_EDIT
admin-api	GET	`/api/segments/fields`	VIEWER
admin-api	POST	`/api/segments/preview`	CAMPAIGN_EDIT
admin-api	GET	`/api/segments/{id}`	VIEWER
admin-api	PUT	`/api/segments/{id}`	CAMPAIGN_EDIT
admin-api	POST	`/api/segments/{id}/size`	CAMPAIGN_EDIT
admin-api	GET	`/api/support-requests`	VIEWER
admin-api	POST	`/api/support-requests/{id}/reply`	ANALYST
admin-api	POST	`/api/support-requests/{id}/resolve`	ANALYST
admin-api	GET	`/api/templates`	VIEWER
ingest-api	GET	`/brand/{file}`	public / signed
ingest-api	GET	`/media/{file}`	public / signed
ingest-api	GET	`/meta/webhook`	public / signed
ingest-api	POST	`/meta/webhook`	public / signed
ingest-api	POST	`/pixel/events`	public / signed
ingest-api	POST	`/shopify/proxy/cart`	public / signed
ingest-api	POST	`/shopify/proxy/engagement`	public / signed
ingest-api	POST	`/shopify/proxy/notify-me`	public / signed
ingest-api	GET	`/shopify/proxy/pay/{ref}`	public / signed
ingest-api	POST	`/shopify/proxy/prompt-event`	public / signed
ingest-api	POST	`/shopify/proxy/refresh`	public / signed
ingest-api	POST	`/shopify/proxy/register`	public / signed
ingest-api	GET	`/shopify/proxy/sw.js`	public / signed
ingest-api	GET	`/shopify/proxy/t/{ref}`	public / signed
ingest-api	POST	`/shopify/proxy/unregister`	public / signed
ingest-api	POST	`/shopify/thankyou/whatsapp-optin`	public / signed

Service	Method	Path	Role
ingest-api	GET	`/u/{token}`	public / signed
ingest-api	POST	`/u/{token}`	public / signed
ingest-api	POST	`/webhooks/courier`	public / signed
ingest-api	POST	`/webhooks/razorpay`	public / signed
ingest-api	POST	`/webhooks/ses`	public / signed
ingest-api	POST	`/webhooks/shopify`	public / signed
ingest-api	POST	`/webhooks/sms`	public / signed

14. Data model

PostgreSQL, Flyway migrations. Main tables: `identities, identity_keys, profiles, consents` (who and what they agreed to); `webhook_inbox, events` (what happened); `carts, checkouts, orders, order_lines, shipments, payment_attempts, products` (store state); `cascade_runs, cascade_attempts, sends, conversions, holdouts` (messaging); `campaigns, campaign_recipients, segments, media`; `config_keys, config_versions, config_proposals`; `operators, audit_log, pii_unmask_log, privacy_requests, support_requests`.

Migration	Purpose
`V1__core.sql`	V1: core data spine — identity graph, events, consent ledger, sends, money. Postgres 16. gen_random_uuid() is built in (13+), so no pgcrypto needed. Three rules this schema encodes: * phone is the primary identity key in India, stored as 91XXXXXXXXXX * consents is APPEND-ONLY; current state is a view over history * sends records BLOCKED and DEFERRED attempts too: that table is the audit trail and the analytics deno
`V2__admin_and_config.sql`	V2: admin console — operators, RBAC, versioned config, campaigns, audit. Builds on V1 (identities, profiles, consents, sends, journey_runs, ...). The one non-obvious decision in this file: config is APPEND-ONLY and versioned, exactly like the consent ledger. Settings that govern whether a message may be sent are evidence, not preferences. When someone asks "why did this customer get a marketing WhatsApp at 22:40",
`V3__shopify_and_push.sql`	V3: Shopify storefront + FCM web push. See docs/technical/phase-1-core-platform.md §1 and phase-2-shopify-web-push.md.
`V4__orchestrator.sql`	V4: orchestrator — cascades, WhatsApp capability, consent copy registry, Meta template state, campaign rate buckets. See CLAUDE.md §3–4 and docs/technical/phase-4, phase-5.
`V5__payments_razorpay.sql`	V5: payment attempts from Razorpay webhooks. payment_failed is a UTILITY WhatsApp template only when it references a real, gateway-confirmed payment attempt. This table is that evidence. A checkout that stalls with no row here is an abandoned checkout (MARKETING) instead. See docs/technical/phase-5-journeys.md §2. Razorpay amounts arrive in the smallest currency unit, i.e. paise for INR, which is exactly our storag
`V6__identity_functions.sql`	V6: identity resolution and merge, as database functions. Why in the database: resolution must be atomic across concurrent webhooks. Shopify sends checkouts/update and orders/create for the same shopper within milliseconds, often to different pods. Advisory locks on the keys serialise exactly the resolutions that touch the same phone/customer/email, and nothing else, so two pods can never mint two identities for one
`V7__push_copy_v1.sql`	V7: register the storefront push soft-ask copy (phase-2 §3). A push grant covers exactly what the shopper was shown, and the server refuses a registration whose copy version (or wording) is not registered here. The text must match the theme embed's "Push soft-ask text" default byte for byte. New words = a new version (rows are immutable, V4); register it here or in the admin console before the theme shows it.
`V8__dispatch_and_kill_switches.sql`	V8: event dispatch claims, config defaults, kill switches (P3-T02). Planned as V10. V8 and V9 (P1 gaps, P2 views) had not been built, and a database at V10 would refuse a later V8 or V9, so this took the next free number and those moved up (docs/implementation/README.md, Migrations).
`V9__shipments_and_order_lifecycle.sql`	V9: P1 gaps — shipments and courier events (P1-T02), order cancellation and refunds (P1-T04), per-location inventory (P1-T03), last-seen prices (P1-T04). One migration for all of P1's gap tasks. Added with P1-T03; the shipment and refund tables wait for their handlers.
`V10__pixel_session_index.sql`	browse_abandon (P3-T07) reads one browser's recent pixel events on every product view to find its session (views separated by < 30 min idle). Pixel events carry no identity, so the browser's client_id is the key.
`V11__push_funnel_views.sql`	P2 views (phase-2 §6 and §9). Planned as V10; the pixel index took V10.

Migration	Purpose
`V12__thankyou_whatsapp_copy.sql`	V12: WhatsApp consent copy for checkout (P2-T05). Rows are immutable (V4): new words = a new version, registered before the storefront shows them. Two bases, as most Indian D2C stores run it: checkout_notice_v1: order and delivery updates only. The notice is the checkout phone field's label (Shopify admin -> checkout language), shown before the number is typed; nobody ticks anything. DPDP Act s.7(a): a number given
`V13__push_copy_v2.sql`	V13: push soft-ask copy v2 (asked early: 2nd page view or ~20 s on site, as well as at add-to-cart and notify-me). The browser's own "Allow" still has to follow the tap, so push is never assumed. v2 names order updates, so a grant covers transactional and marketing. push_v1 stays registered: themes that saved its text keep working. The text must match the theme embed's "Push soft-ask text" byte for byte.
`V14__whatsapp_optin_copy.sql`	V14: the remaining WhatsApp opt-in copy (P2-T07). Both are explicit actions by the shopper, so both cover offers as well as order updates; order updates alone come from the checkout notice (checkout_notice_v1, V12). wa_v1: the cart page checkbox (app block "Engage WhatsApp opt-in"), never pre-ticked. The label is the theme embed's "WhatsApp opt-in text". wa_inthread_v1: the message the shopper sends us from the wa.m
`V15__checkout_notice_v2.sql`	V15: checkout notice v2 also names offers (decided 2026-10-01, the common Indian D2C approach: WhatsApp marketing to every buyer by default, opt-out). The checkout phone field's label must read exactly this text. From CHECKOUT_NOTICE_SINCE, an order with a phone records WhatsApp order updates and offers on this notice: evidence says basis 'notice_opt_out' for offers (no affirmative action was taken) and is never wri
`V16__dashboard_views.sql`	V16: P6-T02 dashboard views. The admin dashboard reads these, never the raw tables, so the numbers an operator sees during an incident are defined once, here. Built before P4, so the planned P4–P7 migrations moved up by one.
`V17__config_key_schemas.sql`	V17: P6-T03 validation bounds for every config key. The config API checks each write against value_type and this json_schema (a supported subset: type, minimum, maximum, enum, pattern, minLength, maxLength); a keyword it does not know fails the write rather than being ignored. Platform limits are encoded as validation, not documentation (02-config-and-settings.md): Meta's per-user marketing limit caps WhatsApp marke
`V18__pii_unmask_log.sql`	V18: P6-T04 pii_unmask_log, needed by the journey inspector. Planned for the P6 admin-security migration, which keeps exports and recovery codes (P6-T08). One row per lookup or unmask of a customer identifier by an operator: who, whose, which field and why. It is also what the 50-per-operator-per-day unmask limit (07-api-contract) counts. A lookup that matched nobody is logged too (identity_id NULL): probing numbers
`V19__segment_sources.sql`	V19: P6-T05 data the segment predicates read that nothing stored before. order_lines: what each order contained (bought_product, bought_size, bought_product_type). Written by ingest from order payloads. products: product type, tags and which option is the size, from products/update. The order payload carries none of these. Customer tags go into profiles.attrs.shopify_tags (ingest; no DDL). Backfilled below from
`V20__campaign_followup.sql`	V20: P6-T06 campaigns. A campaign is run as a one-off cascade (intent campaign:<id>): its channel and template, then optionally a follow-up on another channel some hours later to those who did not click (phase-6 §4: push first, WhatsApp 6 h later).
`V21__admin_security.sql`	V21: P6-T08 admin security: MFA recovery codes and exports. (pii_unmask_log came earlier, in V18, with the journey inspector.)
`V22__wa_status_and_inbound.sql`	V22: P4-T04 WhatsApp statuses and inbound messages.
`V23__whatsapp_numbers.sql`	V23: P4-T03 WhatsApp number health, and the WhatsApp config keys. (Planned as the P4 number-health migration; built after V22.)

Migration	Purpose
`V24__sms_rate.sql`	V24: P4-T06 SMS (MSG91, DLT). What one transactional SMS costs, so policy books it in spend_ledger at send time (SendRepository.markSent) and a daily budget can cap it. Delivery reports land in webhook_inbox as source 'msg91'.
`V25__abandon_journey_config.sql`	V25: abandoned checkout and cart recovery on WhatsApp (P5-T03/T04, the owner's choice of 2026-10-10: do what most Indian D2C brands do). A WhatsApp reminder first, push as the free fallback and second nudge, and a last WhatsApp with a discount code only if an operator has set one. No SMS (CLAUDE.md §3.2: no promotional SMS). Takes the cart and checkout keys from the planned P5-T01 journey migration; that migration k
`V26__abandon_discount_limits.sql`	V26: who may get the abandoned-checkout discount (owner's decision, 2026-10-10). A shopper who learns that leaving checkout earns a code waits for it every time, so the discount goes at most once per customer per cooldown, and optionally only to people who have never ordered. Pair it with "Limit to one use per customer" on the code in Shopify admin.
`V27__abandon_unique_codes.sql`	V27: single-use abandoned-checkout codes (owner's decision, 2026-10-11). In unique mode the worker creates one Shopify code per customer when the discount WhatsApp goes out: one use, valid discount_valid_hours. Needs the write_discounts scope on the store's app; without it the discount message is skipped and the reminders still go. Shared mode keeps the operator's code (journey.abandon.discount_code / _label).
`V28__winback_config.sql`	V28: win-back, "long time no see" (P5-T08; owner's decision 2026-10-11: do what most Indian D2C brands do). A customer whose last order is lapsed_days old and who has not been on the site for no_visit_days gets a warm "here's what's new" WhatsApp, a push a week later, and an offer a week after that only if a discount is set. Once per lapse; any order ends it. Takes the winback keys from the planned P5-T01 migration.
`V29__journey_collisions.sql`	V29: journeys collide by priority (P5-T01, phase-5 §5). A step is blocked HIGHER_PRIORITY_ACTIVE while the same person has a more important journey (priority 2 or 3) live, or ended less than suppress_hours ago. Order updates (priority 1) never block and are never blocked.
`V30__payment_failed_config.sql`	V30: payment_failed (P5-T02). How long after a gateway-confirmed failure the first WhatsApp goes: most UPI failures are retried at once, and a message too early reads as the brand not knowing. Taken from the planned P5-T01 journey migration.
`V31__post_purchase_config.sql`	V31: post_purchase (P5-T06): how many days after delivery the "how do you like it?" WhatsApp goes (at 11:00 IST).
`V32__journey_holdouts.sql`	V32: holdouts on every journey (P5-T09). Each journey's key is its holdout experiment; 10% of people are kept out of each marketing journey so its lift can be measured (P7). The policy engine applies it to marketing sends only, so transactional journeys (order updates, payment_failed) are never held out, and campaigns keep their own control group. A journey can be set differently in the admin console with the journey
`V33__support_requests.sql`	V33: support requests (owner's decision 2026-10-15: failed deliveries are handled by staff, not forwarded to the courier automatically). Our WhatsApp runs on the Cloud API, which has no inbox: a customer's reply only reaches us, so staff see it here. delivery a WhatsApp reply within 72 h of the "delivery did not go through" message (ndr_raised_v1), or its reschedule button. One open request per parcel; later
`V34__pii_field_support_reply.sql`	V34: reading a support request's reply text (V33) in the console is a PII access like a phone or an address: it is logged in pii_unmask_log as field 'support_reply' and counts toward the operator's daily limit.
`V35__measurement.sql`	V35: measurement (P7-T03). Lift of each journey against its holdout, what messaging cost per intent, and the monthly check of our WhatsApp spend against Meta's bill.

Migration	Purpose
`V36__privacy.sql`	V36: privacy (P7-T05). DPDP erasure and Shopify's compliance webhooks. erase_identity() removes a person: the identity, everything merged into it, and every copy of their phone, email and order data outside the identity graph (orders, checkouts, payments, raw webhook payloads, courier events, exports). What must still add up afterwards (spend, conversions) is moved to a tombstone identity that identifies nobody. aud
`V37__shared_rate_limits.sql`	V37: storefront rate limits shared across ingest pods (P7-T06). Production runs 3–10 ingest pods; per-pod counters would let N times the limit through. UNLOGGED: a crash loses the current windows, which only forgives a few requests; in exchange every hit is a cheap write with no WAL.
`V38__push_images.sql`	V38: pictures in push notifications. products.image_url: the product's main photo from products/update, on Shopify's CDN (public). Journey pushes show it as the large image.
`V39__campaign_review.sql`	V39: every campaign is reviewed before it can go out (owner's decision 2026-10-15). After its dry run a campaign waits for review; an operator with sending rights checks the preview, the audience and the cost and approves it, and only then can it be armed. One login is enough: the author may approve their own campaign unless approval.four_eyes is switched on.
`V40__self_approved_config.sql`	V40: one login can make a CRITICAL settings change (owner's decision 2026-10-15). It still goes through a proposal (the review step), and the proposer may approve it themselves while approval.four_eyes is off. The database keeps the four-eyes rule unless the decision is explicitly marked self-approved, which the service does only when the setting allows it.
`V41__attribution.sql`	V41: which message an order is credited to, for the Analytics reports. Last touch, two windows, both settings: click: the latest message the customer clicked in the attribution.click_days before the order (push clicks are tracked); view: otherwise the latest message delivered or read in the attribution.view_hours before it (WhatsApp buttons are not tracked, so a read WhatsApp is credited this way). Messages that on

15. Operations

Scheduled jobs

Service	Job	When	What
ingest-api	InboxProcessor	every 2 s	Processes stored webhooks (retries with back-off, dead-letters after the last attempt).
ingest-api	InboxHousekeeping	every 15 s / 03:00 IST	Inbox gauges; deletes processed webhooks older than 7 days.
ingest-api	RateLimiter	hourly :17	Purges storefront rate windows older than a day.
worker	WorkerJobs.dispatchEvents	every 1 s	Turns events into journey intents.
worker	WorkerJobs.tickCascades	every 5 s	Runs journey steps that are due (deferrals, delays).
worker	WorkerJobs.runCampaigns	every 2 s	Sends armed campaigns in rate-limited batches.
worker	QueuedSweeper	every 1 min	Marks sends stuck in queued as lost in flight.
worker	PushTokenGauge	every 60 s	Push subscriber gauges.
worker	WorkerJobs.findLapsedCustomers	01:30 IST	Win-back: emits became_lapsed.
worker	WorkerJobs.clearResolvedSupportText	02:00 IST	Clears support reply text 30 days after it is resolved.
worker	TokenLifecycle	02:30 IST	Deactivates push tokens not refreshed for 180 days.
worker	WorkerJobs.recheckCapability	03:30 IST	INCAPABLE WhatsApp numbers go back to UNKNOWN after 120 days.
worker	WorkerJobs.reconcileSpend	06:00 IST, 1st–15th	Last month's WhatsApp ledger against Meta's uploaded bill; alerts over 1%.
worker	WorkerJobs.syncWhatsAppTemplates	every 30 min	Meta template statuses, categories and quality; number quality.
admin-api	ExportHousekeeping	every 15 min	Expires export files after 24 h.

Environments

Environment	Where	State
Local	Developer PC: Gradle, local PostgreSQL (dev and test databases), Angular dev server	All tests: 678 backend, 45 console, 19 database invariants
Dev	engage-dev.wumika.com (46.202.164.251), Docker; deploys on every push to `dev` (tests, images, start)	Dev store, Razorpay test, push only; running the build from 7 days ago (V21) until the next push
Staging / Production	AWS (planned)	Not set up

Monitoring

Prometheus metrics at `/prometheus` (token): webhooks received by result, inbox backlog and oldest pending, handler errors, push tokens, Hikari pool, WhatsApp category drift. Errors are logged with references; forwarding alerts to a person is

still to be set up.

Load test

`load/k6/webhooks.js` replays signed traffic at up to 400 events/s (refuses production hosts); `load/k6/after.sh` checks the inbox drains within 5 minutes, no pool timeouts and journey lag under 30 s. Results: `docs/qa/p7-load.md`.

Runbooks

`docs/08-deployment-and-ops.md`: quality rating drops, spend spike, shop redact, DPDP obligations, backup and recovery.

16. Status and pending work

Area	State	To go live
Push notifications (web)	Built and live on the dev store	Live store: deploy the push embed and the App Proxy at go-live.
WhatsApp	Built and tested against a simulated Meta	Submit the 19 templates; Indian number and business verification; WhatsApp settings on the server; Meta webhook URL.
SMS (MSG91, DLT)	Built and tested against a simulated MSG91	MSG91 account; DLT entity, sender ID and the 4 templates (IDs are PENDING).
Email	Built (SES) but switched off	Owner's decision: email stays with Shopify Email (turn on its abandoned-checkout email).
Shopify (dev store)	Connected: all scopes granted, web pixel connected	Live app: production webhook URL, full topics and scopes, App Proxy, extensions.
Razorpay	Built; test mode on dev	Live webhook to production; ADR-002 check on Shopify refs in notes.
Shiprocket (courier)	Built	COURIER_WEBHOOK_TOKEN and the webhook in Shiprocket.
Discount codes	Built; proven on the dev store	Live app needs write_discounts approved.
Admin console	Built	—
Analytics and reports	Built	—
Privacy (DPDP, Shopify compliance)	Built; compliance webhooks on the dev app only	Add compliance topics to the live app at go-live.
Load test, pen test, rotation, backups	Scripts and checklists ready	Needs staging and production (AWS).
Loyalty points	Not built (parked)	Planned as P8–P11.

Before production

1. Deploy the current code to dev (push `dev`: migrations V22–V41).
2. WhatsApp: submit the 19 templates; on dev add the WhatsApp settings, point Meta's webhook at engage-dev, add test recipients, and narrow the dev allowlist to the approved test customer first.
3. Shiprocket courier webhook and token; MSG91 account and DLT registration.
4. Alert forwarding (errors to a person); `CHECKOUT_NOTICE_SINCE`; verify the push embed, thank-you block and Razorpay test webhook on the dev store; Shopify Email's abandoned-checkout email.

Production (AWS)

1. Live Shopify app: production webhook URL, all topics and scopes, App Proxy, extensions, compliance topics.
2. Production secrets and tokens, rotated; `PUBLIC_BASE_URL`; migration job; backups with a rehearsed restore.
3. Providers pointed at production (Meta, Razorpay live, MSG91, Shiprocket, Firebase).
4. Staging load test, external pen test, then go-live: open the allowlist (`*`) last.